

FleetSpark — Deployment Guide

Version: 1.0

Date: June 11, 2026

Target: Coolify on EC2 (Ubuntu 24 LTS)

1. Prerequisites

- EC2 instance running (t3.medium minimum, t3.large recommended)
 - Coolify installed on the instance
 - Domain pointed to the server IP
 - GitHub repository created
 - Docker Hub or GitHub Container Registry account
-

2. Server Setup

Coolify Installation (if not already installed)

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash
```

Required Services on the Server

- Docker & Docker Compose
 - Nginx/Traefik (via Coolify)
 - GitHub CLI (`gh`)
 - Node.js 22 (for local tooling)
 - PostgreSQL client (`psql`)
-

3. GitHub Repository Setup

Create Repository

```
# On GitHub, create a new private repo: fleetspark/fleetspark

# Initialize locally
git init
git remote add origin git@github.com:syncsphere7/fleetspark.git

# Add all project files
git add .
git commit -m "Initial commit: FleetSpark MVP"
git push -u origin main
```

Branch Strategy

Branch	Purpose	Deploy
main	Production	Auto-deploy to production
develop	Development	Auto-deploy to staging
feature/*	Feature branches	No deploy

4. Coolify Project Setup

Step 1: Create Project in Coolify UI

1. Open Coolify dashboard: `http://<server-ip>:8000`
2. Click "New Project"
3. Name: `FleetSpark`
4. Description: `EV Fleet Management Platform`

Step 2: Add GitHub Integration

1. Settings → Git Sources → GitHub
2. Connect GitHub account (use `SyncSphere7` org)
3. Install GitHub App on the repository

Step 3: Create Production Resource

1. Inside FleetSpark project, click "New Resource"
2. Select "Application"
3. Source: GitHub → `syncsphere7/fleetspark`
4. Branch: `main`
5. Build Pack: Dockerfile
6. Dockerfile location: `/Dockerfile`
7. Health check: `GET /health` on port 3000

Step 4: Configure Domain

1. In resource settings, go to Domains
2. Add: `fleetspark.syncspherellc.com`
3. SSL: Enabled (Let's Encrypt auto)
4. Click "Save" — Coolify provisions SSL automatically

Step 5: Configure Ports

- Container port: `3000`
- Exposed port: `443` (HTTPS via Traefik)

5. Environment Variables

Set these in Coolify UI (Settings → Environment Variables):

Application

```
NODE_ENV=production
PORT=3000
CLIENT_URL=https://fleetspark.syncspherellc.com
```

Database (Coolify-managed PostgreSQL)

```
DATABASE_URL=postgresql://fleetspark:{{password}}@fleetspark-db:5432/fleetspark
```

Note: If using Coolify's managed database, replace `fleetspark-db` with the service alias.

Redis

```
REDIS_URL=redis://fleetspark-redis:6379
```

JWT

```
JWT_SECRET=<generate: openssl rand -hex 32>
JWT_REFRESH_SECRET=<generate: openssl rand -hex 32>
JWT_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
```

Email (Resend)

```
RESEND_API_KEY=re_<from_resend_dashboard>
EMAIL_FROM=FleetSpark <noreply@fleetspark.syncspherellc.com>
```

SMS (Twilio)

```
TWILIO_ACCOUNT_SID=AC<from_twilio_console>  
TWILIO_AUTH_TOKEN=<from_twilio_console>  
TWILIO_FROM_NUMBER=+18456134631
```

File Storage (MinIO)

```
S3_ENDPOINT=http://fleetspark-minio:9000  
S3_ACCESS_KEY=<generate>  
S3_SECRET_KEY=<generate>  
S3_BUCKET=fleetspark
```

Telemetry

```
TELEMETRY_API_KEY=<generate: openssl rand -hex 24>  
TELEMETRY_THROTTLE_MS=30000
```

Defaults

```
DEFAULT_DIESEL_PRICE_UGX=5500  
DEFAULT_CO2_FACTOR_KG_PER_LITER=2.68
```

6. Dockerfile

```
# fleetspark/Dockerfile

# Stage 1: Build
FROM node:22-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./
COPY prisma ./prisma/

# Install dependencies
RUN npm ci

# Generate Prisma client
RUN npx prisma generate

# Copy source
COPY . .

# Build client (Vite)
RUN npm run build:client

# Build server (if using tsc)
RUN npm run build:server

# Stage 2: Production
FROM node:22-alpine AS production

WORKDIR /app

# Install production dependencies only
COPY package*.json ./
RUN npm ci --production

# Copy built assets
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/client/dist ./client/dist
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma

# Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

EXPOSE 3000

CMD ["npm", "start"]
```

7. GitHub Actions CI/CD

```
# .github/workflows/deploy.yml

name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: 'npm'

      - run: npm ci

      - name: Run tests
        run: npm test

      - name: Run linter
        run: npm run lint

  build-and-deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: |
          docker build -t syncsphere7/fleetspark:${{ github.sha }} .
          docker tag syncsphere7/fleetspark:${{ github.sha }} syncsphere7/fleetspark:latest

      - name: Push to GitHub Container Registry
        run: |
          echo ${ secrets.GHCR_TOKEN } | docker login ghcr.io -u SyncSphere7 --password-stdin
          docker push syncsphere7/fleetspark:${{ github.sha }}
          docker push syncsphere7/fleetspark:latest

      - name: Trigger Coolify Deploy
        run: |
          curl -X POST "${ secrets.COOLIFY_WEBHOOK_URL }" \
            -H "Content-Type: application/json" \
            -d '{"sha": "${{ github.sha }}"}'
```

GitHub Secrets to Set

Secret	Value
GHCR_TOKEN	GitHub Personal Access Token with <code>write:packages</code> scope
COOLIFY_WEBHOOK_URL	From Coolify resource settings → Webhook

8. Database Migration Strategy

Initial Setup

```
# Connect to production DB via Coolify terminal or SSH
npx prisma migrate deploy
npx prisma db seed # Only for first deploy
```

Ongoing Migrations

- Migrations run automatically on deploy via `postdeploy` script in Dockerfile or Coolify's deploy hook
- Add to Dockerfile CMD or use Coolify's "Execute Command" post-deploy hook:

```
npx prisma migrate deploy
```

Backup Strategy

```
# Daily backup via cron on the server
0 2 * * * pg_dump -U fleetspark -d fleetspark | gzip > /backups/fleetspark-$(date +%Y%m%d).sql.gz

# Keep last 30 days
find /backups -name "fleetspark-*.sql.gz" -mtime +30 -delete
```

9. Monitoring Setup

Health Check Endpoint

```
// src/server/routes/health.routes.ts
app.get('/health', (req, res) => {
  res.json({
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    environment: process.env.NODE_ENV,
  });
});
```

Uptime Kuma (Self-hosted)

Deploy via Coolify: Add resource → Docker Image → `louislam/uptime-kuma`

- Monitor `https://fleetspark.syncspherellc.com/health`

- Alert to Telegram/SMS on downtime

Log Management

```
// pino logger config
{
  "level": "info",
  "transport": {
    "target": "pino-pretty",
    "options": { "colorize": true }
  },
  // In production, log to stdout (Docker captures it)
}
```

10. Rollback Procedure

Via Coolify UI

1. Go to Resource → Deployments
2. Find previous successful deployment
3. Click "Redeploy"

Via Git

```
# Revert last commit
git revert HEAD
git push origin main

# Or reset to specific commit
git reset --hard <commit-sha>
git push origin main --force # Use with caution
```

Database Rollback

```
# List migrations
npx prisma migrate status

# Rollback specific migration (manual SQL needed)
# Always backup before rollback
pg_dump -U fleetspark -d fleetspark > pre-rollback-backup.sql

# Then run reverse SQL manually
```


11. Scaling Checklist

When to Scale

Signal	Action
CPU > 70% for 5 min	Upgrade EC2 instance
- Response time p95 > 1s	Add read replica for DB
DB disk > 80%	Add storage or archive old data
Redis memory > 80%	Increase Redis memory limit
>50 concurrent WebSocket connections	Add WebSocket server

Vertical Scaling

1. In AWS Console, stop EC2
2. Change instance type (t3.medium → t3.large)
3. Start EC2
4. Coolify auto-detects and resumes

Horizontal Scaling (Future)

- Add second API server behind Traefik
- Add PostgreSQL read replica
- Use Redis Sentinel for HA

12. Security Hardening

Server Level

```
# UFW firewall (via Coolify, already configured)
ufw allow 22      # SSH
ufw allow 80      # HTTP
ufw allow 443     # HTTPS

# Automatic security updates
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

Application Level

- Helmet.js headers enabled
- Rate limiting on all endpoints
- CORS restricted to CLIENT_URL
- SQL injection prevented by Prisma parameterized queries
- XSS prevented by React auto-escaping + input validation

Regular Security Tasks

Task	Frequency
Dependency audit (<code>npm audit</code>)	Weekly
SSL cert check	Monthly
Access log review	Monthly
Rotate JWT secrets	Quarterly
Review user access	Quarterly

13. Disaster Recovery

Scenario: Server Failure

1. New EC2 instance
2. Install Coolify
3. Restore database from latest backup
4. Deploy from GitHub (all config in env vars)
5. Update DNS if IP changed

Scenario: Database Corruption

1. Stop application
2. Restore from latest backup: `gunzip < backup.sql.gz | psql -U fleetspark`
3. Verify data integrity
4. Restart application

Scenario: Accidental Data Deletion

1. If soft-delete: `UPDATE ... SET is_active = true`
2. If hard-delete: restore from backup to staging, export affected records, import to production

14. Staging Environment

Setup

1. Create `develop` branch
2. In Coolify, create new Resource pointing to `develop` branch
3. Domain: `staging.fleetspark.syncsphere11c.com`
4. Use separate PostgreSQL instance (Coolify manages this)
5. Use separate Redis instance

Staging vs Production

Setting	Staging	Production
Branch	deve lop	main
Domain	staging.fleetspark...	fleetspark...
Database	fleetspark_staging	fleetspark
Email	Resend test mode	Resend live
SMS	Twilio test creds	Twilio live
NODE_ENV	staging	production